Mohó algoritmusok

Elmélet, bizonyítások és klasszikus versenyfeladatok

Szerző: Nikházy László
Előadó: Gyimesi Péter

2026. január 21.

Horváth Gyula és Zsakó László diasorainak felhasználásával

Mit nevezünk a mohó algoritmusnak?

Alapelv: A megoldást lépésről lépésre építjük fel, és minden lépésben egy *lokálisan optimális* döntést hozunk.

Kulcskérdés: Miért vezetnek ezek a lokális döntések globálisan optimális megoldáshoz?

- Általában többféleképpen is lehet lokálisan optimális döntést hozni, és nem nehéz kitalálni ilyen stratégiákat.
- Így a helyesség **bizonyítása** valójában fontosabb, mint maga az algoritmus.

A mohó stratégia elemei

1. Fogalmazzuk meg az optimalizációs feladatot úgy, hogy választások sorozatával építjük fel a megoldást!
2. Mohó választási tulajdonság: Mutassuk meg, hogy mindig van olyan megoldása az eredeti feladatnak, amely a mohó választással kezdődik!
3. Optimális részprobléma tulajdonság: Bizonyítsuk be, hogy a mohó választással olyan redukált problémát kapunk, amelynek optimális megoldásához hozzávéve a mohó választást, az eredeti probléma megoldását kapjuk!

— Horváth Gyula, Zsakó László

Bizonyítási technikák

Kivágás - beillesztés / Exchange Argument

- Tegyük fel, hogy van egy optimális megoldás, amely eltér a mohó választástól.
- Megmutatjuk, hogy az eltérő elemek *kicserélhetők* anélkül, hogy a megoldás romlana.

Előnyben maradás / Staying Ahead Argument

- Összehasonlítjuk a mohó megoldást bármely más részleges megoldással.
- Bizonyítjuk, hogy a mohó megoldás minden lépés után „legalább olyan jó”.

Példa feladatok

Klasszikus, tanulságos példákon mutatjuk be a *bizonyítási technikákat*

Movie Festival (CSES)

Adott $\mathtt{n}$ ($1 \leq \mathtt{n} \leq 2 \cdot 10^5$) film vetítési ideje, mindegyik egy intervallummal:

$$(\mathtt{l_i}, \mathtt{r_i}) \qquad (1 \leq \mathtt{l_i} < \mathtt{r_i} \leq 10^9)\,.$$

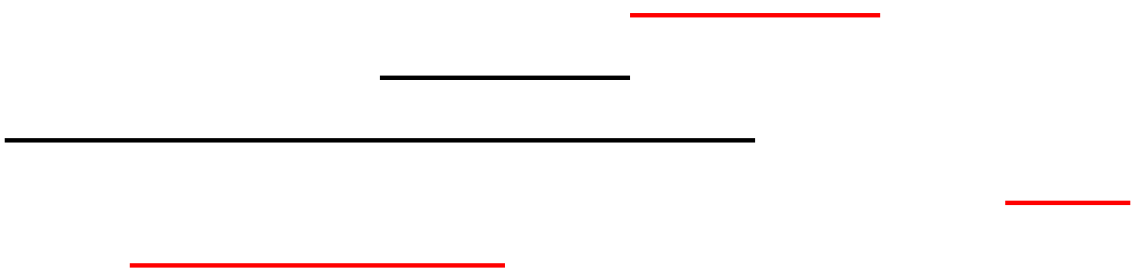
Egyszerre csak **egy filmet** tudsz nézni, egy film teljes hosszában leköti az idődet.

Feladat: Válaszd ki a lehető legtöbb filmet úgy, hogy a kiválasztott filmek időintervallumai **ne fedjék egymást**.

Példa:

Filmek: (5, 7), (3, 5), (0, 6), (1, 4), (8, 9)

Egy optimális választás: (1, 4), (5, 7), (8, 9)



Movie Festival — Mohó megoldás

Mohó stratégia:

- Rendezzük a filmeket növekvő **befejezési idő** szerint.
- Sorban végigmegyünk rajtuk, és mindig kiválasztjuk a legkorábban befejeződő filmet, amely kompatibilis az eddigiekkel.

Miért optimális a megoldás, amit kapunk?

Movie Festival — Helyesség bizonyítása

1. Mohó választási tulajdonság

Létezik optimális megoldás, amelyben az első film a legkorábban befejeződő film.

Indoklás: Kivágás - beillesztéssel (Exchange Argument)

- Legyen $\mathtt{F}$ a legkorábban befejeződő film.
- Vegyünk egy optimális megoldást, amelyben az első film $\mathtt{G} \neq \mathtt{F}$.
- Mivel $\mathtt{r_F} \leq \mathtt{r_G}$, $\mathtt{G}$ lecserélhető $\mathtt{F}$-re anélkül, hogy a további filmek közül bármelyikkel ütközne.

2. Optimális részprobléma tulajdonság

A fennmaradó probléma ugyanolyan szerkezetű, mint az eredeti.

Fennmaradó probléma: kihagyjuk az összes filmet, ami ütközik a legkorábban befejeződővel - hiszen ezek közül egyik sem lehet benne a megoldásban. A megmaradó filmekkel teljesen ugyanaz a feladat.

A többi filmnek bármilyen optimális megoldásához hozzávesszük a legkorábban befejeződő filmet, akkor szükségképpen optimális megoldást kapunk az eredeti feladatra.

Towers (CSES)

Egymás után érkezik n kocka, melyeknek méretei:

$$a_1, a_2, \ldots, a_n \qquad (1 \leq n \leq 2 \cdot 10^5,\ 1 \leq a_i \leq 10^9).$$

A kockákat minél kevesebb toronyba kell összepakolni. Egy kocka csak nála nagyobb kockára tehető rá. Amikor érkezik egy kocka, rögtön rá kell rakni egy másikra, vagy új tornyot kezdeni. Minimum hány toronyra van szükség?

Példa:

Sorozat: 3, 8, 2, 1, 5

Egy optimális elhelyezés:

- Torony 1: 3, 2, 1
- Torony 2: 8, 5

Szükséges tornyok száma: 2

Towers — Mohó megoldás

Megfigyelés: Ha egy elem több torony tetejére is rátehető, akkor érdemes azt választani, amelynek a teteje a lehető **legkisebb, de még nagyobb** az aktuális elemnél.

Mohó stratégia:

- Balról jobbra dolgozzuk fel a sorozatot.
- Fenntartjuk a tornyok tetejének értékeit.
- Az aktuális a_i elemet:
  - arra a toronyra tesszük, amelynek teteje a legkisebb olyan érték, amely $> a_i$;
  - ha nincs ilyen torony, új tornyot kezdünk.
- Hatékony megvalósítás: bináris kereséssel
  - Vegyük észre, hogy a tornyok tetejei mindvégig monoton növekvő sorozatot alkotnak.

Miért vezet ez minimális számú toronyhoz?

1. Előnyben maradás tulajdonság

A mohó algoritmus minden lépés után a lehető legjobb helyzetet tartja fenn.

Egy konkrét példán: amikor a `3, 8` után jön a 2-es kocka, három lehetőségünk is van, amely után a tornyok tetejei lehetnek: `(2, 8)`, `(3, 2)` és `(3, 8, 2)`. Látható, hogy:

- `(2, 8)` jobb, mint a `(3, 2)`, hiszen minden ami rárakható a 3-asra, a 8-asra is rárakható, sőt még több is.
- `(2, 8)` jobb, mint `(3, 8, 2)`, mert igazából a harmadik helyre odaképzelhetünk egy $+\infty$ tetejű tornyot (az üres helyre bármi tehető), és átrendezve: `(2, 8,` $+\infty$`)` jobb, mint `(2, 3, 8)`, mert minden pozícióban $\geq$ kocka van a tetején.

Általánosan:

- Vizsgáljuk az első `k` elemet.
- Tekintsük a mohó megoldás tornyainak tetejét növekvő sorrendben.
- Bármely más elhelyezés esetén a tornyok teteje növekvő sorrendben minden pozícióban kisebb vagy egyenlő (azonos hosszúra kiegészítve őket, az üres helyeket $+\infty$-nek tekintve).

Towers — Helyesség bizonyítása

2. Optimális részprobléma tulajdonság

A feladatra tekinthetünk úgy, hogy van egy kezdeti állapot is megadva, ami a tornyok tetejein lévő kockákból áll, ami eredetileg csak egy üres lista.

Az aktuális elem elhelyezése után a fennmaradó probléma pontosan ugyanolyan szerkezetű. A többi kockát a tornyok tetejei által alkotott kezdeti állapotból kiindulva kell elhelyezni.

Mivel a mohó választás minden lépésben a „legkedvezőbb” állapotot hagyja maga után, a további elemek optimális elhelyezésével kiegészítve optimális megoldást ad.

Towers — Kód

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int main() {
5    int n; cin >> n;
6    vector<int> tops;
7    for (int i = 0; i < n; i++) {
8      int cube; cin >> cube;
9      auto it = upper_bound(tops.begin(), tops.end(), cube);
10     if (it == tops.end()) tops.push_back(cube);
11     else *it = cube;
12   }
13   cout << tops.size();
14 }
```

Tasks and Deadlines (CSES)

Adott n ($1 \leq n \leq 2 \cdot 10^5$) feladat. Minden feladathoz tartozik:

- egy elvégzési idő t_i,
- egy határidő d_i.

A feladatokat **egymás után**, átfedés nélkül végezzük el, tetszőleges sorrendben.

Ha egy feladat a C_i időpontban fejeződik be, akkor a hozzájárulása az összpontszámhoz: $d_i - C_i$.

Feladat: Határozzuk meg a feladatok olyan sorrendjét, amely maximalizálja az

$$\sum_{i=1}^{n} (d_i - C_i)$$

összértéket!

Tasks and Deadlines — Mohó megoldás

Kulcsmegfigyelés: A határidők összege rögzített, ezért az összpontszám akkor maximális, ha a feladatok **minél korábban fejeződnek be**.

Formálisan: $\sum_{i=1}^{n} (d_i - C_i)$ maximalizása ekvivalens a befejezési idők $\sum C_i$ összegének **minimalizálásával**.

Mohó stratégia:

- Rendezzük a feladatokat növekvő **feldolgozási idő** (t_i) szerint.
- Ebben a sorrendben hajtsuk végre őket.

Ez az ún. **Shortest Processing Time First (SPT)** szabály.

Miért garantálja ez a minimális $\sum C_i$ értéket?

Javítás cserével (Exchange Improvement Argument)

Tekintsünk egy tetszőleges feladatsorrendet. Ha ebben szerepel két egymást követő feladat A, B úgy, hogy

$$t_A > t_B,$$

akkor ez egy **inverzió**.

Vizsgáljuk meg, mi történik, ha ezt a két feladatot felcseréljük.

Az A, B sorrendben a két feladat hozzájárulása a befejezési idők összegéhez:

$$(C + t_A) + (C + t_A + t_B).$$

A B, A sorrendben:

$$(C + t_B) + (C + t_B + t_A).$$

Mivel $t_B < t_A$, a csere után a

$$\sum C_i$$

értéke **szigorúan csökken**.

Tehát minden inverzió megszüntetése javít a megoldáson.

Tasks and Deadlines — Következmény

Bármely feladatsorrend véges számú ilyen javító cserével átalakítható a feldolgozási idők szerint növekvő sorrenddé.

Minden egyes csere során a befejezési idők összege csökken, tehát a folyamat nem ronthatja a megoldást.

Ezért a feldolgozási idők szerint rendezett sorrend **globálisan minimális**

$$\sum C_i$$

értéket ad.

Mivel az összpontszám

$$\sum (d_i - C_i)$$

maximalizálása ezzel ekvivalens, a mohó algoritmus optimális.

Room Allocation (CSES)

Adott $\mathtt{n}$ ($1 \leq \mathtt{n} \leq 2 \cdot 10^5$) esemény, mindegyik egy időintervallummal:

$$(\mathtt{l_i}, \mathtt{r_i}) \qquad (1 \leq \mathtt{l_i} < \mathtt{r_i} \leq 10^9)\,.$$

Egy esemény teljes időtartama alatt **egy külön szobát** igényel.

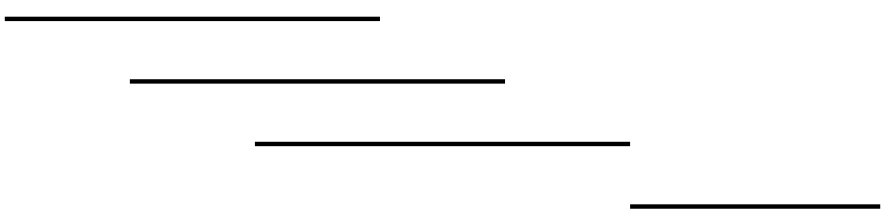
(Alternatív mese: szállodába érkező vendégeknek kell szobákat osztani.)

Feladat: Határozd meg a szükséges szobák **minimális számát**, és adj meg egy hozzárendelést, amely ezt eléri!

Példa:

Események: $(1, 4)$, $(2, 5)$, $(3, 6)$, $(6, 8)$

Minimális szobaszám: 3 Beosztás (szobaszámok): 1, 2, 3, 2



Room Allocation — Mohó megoldás

Megfigyelés: Ha egy esemény kezdésekor van felszabadult szoba, akkor felesleges új szobát nyitni.

Mohó stratégia:

- Rendezzük az eseményeket növekvő **kezdési idő** szerint.
- Tartsuk nyilván, hogy az egyes szobák mikor szabadulnak fel.
- Minden eseménynél: ha van szabad szoba akkor egy tetszőlegesbe tegyük bele az eseményt.
  - Mindegy melyik szobát választjuk, mert a további események nem kezdődhetnek korábban, ugyanúgy bármelyik szobába be lehet őket tenni.
  - A gyakorlatban azt a szobát használjuk, amely **legkorábban** szabadul fel, mert a megvalósítás így könnyű.
  - Adatszerkezet: *prioritási sor*.
- Ha nincs ilyen, új szobát nyitunk.

Miért biztos, hogy így minimális számú szobát használunk?

Room Allocation — Helyesség bizonyítása

Előnyben maradás tulajdonság

A mohó algoritmus minden lépésben a lehető legkevesebb szobát használja az addig feldolgozott eseményekhez.

- Egy új szobát csak akkor nyitunk, ha az aktuális esemény kezdésekor minden meglévő szoba foglalt.
- Ekkor az események száma, amelyek éppen futnak, meghaladja a meglévő szobák számát.

Ez azt jelenti, hogy **bármely megoldásnak** legalább ennyi szobára van szüksége ebben az időpillanatban.

Tulajdonképpen triviális **alsó korlát** a szobák számára az egymást egy pontban átfedő intervallumok száma. A mohó algoritmus pedig ennyi szobát használva megoldja a feladatot, így szükségképpen optimális.

Room Allocation — Kód

```
1  #include <bits/stdc++.h>
2  using namespace std;
3  using pii = pair<int, int>;
4  int main() {
5    int n; cin >> n;
6    vector<array<int, 3>> guests(n);
7    for (int i = 0; i < n; i++) {
8      cin >> guests[i][0] >> guests[i][1];
9      guests[i][2] = i;
10   }
11   sort(guests.begin(), guests.end());
12   vector<int> room(n);
13   priority_queue<pii, vector<pii>, greater<pii>> pq;
14   for (auto [from, to, id] : guests) {
15     room[id] = pq.size() + 1;
16     if (!pq.empty() && pq.top().first < from) {
17       room[id] = pq.top().second;
18       pq.pop();
19     }
20     pq.push({to, room[id]});
21   }
22   cout << pq.size() << "\n";
23   for (int r : room) cout << r << "␣";
24 }
```

Stick Division (CSES)

Adott egy x hosszúságú rúd, amelyet n darab rúdra szeretnénk felosztani, megadott hosszakkal:

$$d_1, d_2, \ldots, d_n, \qquad \sum d_i = x.$$

Egy lépésben bármely rudat kettévághatunk. Egy vágás költsége a szétvágott rúd hossza.

Feladat: Határozzuk meg a darabolás **minimális összköltségét**!

Példa:

$x = 8, \quad d = (2, 3, 3)$

Optimális lépések:

- $8 \to 3 + 5$ (költség: 8)
- $5 \to 2 + 3$ (költség: 5)

Összköltség: $8 + 5 = 13$

Stick Division — Visszafelé gondolkodás

Kulcsötlet: A darabolás helyett gondolkodjunk **visszafelé**: nem vágunk, hanem *összevonunk*.

A végállapotban n különálló darab van:

$$d_1, d_2, \ldots, d_n$$

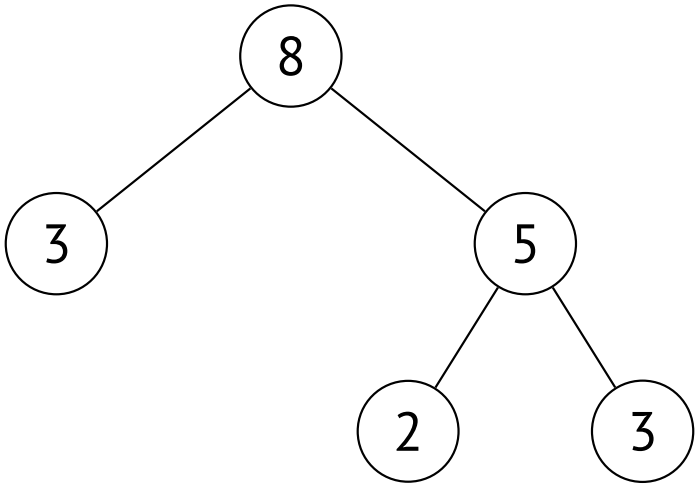
Egy korábbi lépésben két darab **egy közös rúdból származott**, amelynek hossza az összegük volt.

Így a teljes folyamat leírható egy bináris fával:

- a levelek a végső darabok (d_i),
- minden belső csúcs egy összevonás,
- a belső csúcs súlya a két gyerek összhossza,
- az összköltség a belső csúcsok súlyainak összege.

Stick Division — Bináris fa modell

Példa: `d = (2, 3, 3)`



- Belső csúcsok: 8, 5
- Összköltség: $8 + 5 = 13$

Cél: olyan fát találni, amely minimalizálja a belső csúcsok súlyainak összegét.

Stick Division — Mohó megoldás

Az előzőek alapján a feladat: **egy olyan bináris fa felépítése**, amelyben a levelek a d_i hosszúságú darabok, és a belső csúcsok súlyainak összege minimális.

Megfigyelés:

- Egy darab hossza annyiszor járul hozzá az összköltséghez, ahány belső csúcs fölötte helyezkedik el.
- A **mélyebb levelek** többször szerepelnek a költségben.

Következmény:

- hosszú darabokat érdemes *magasabban*,
- rövid darabokat érdemes *mélyebben* elhelyezni a fában.

Mohó stratégia:

- mindig a **két legrövidebb** darabot vonjuk össze,
- ezekből képezzük a fa legmélyebb belső csúcsát.

Ez pontosan a fa *alulról felfelé* történő felépítését jelenti.

Stick Division — Helyesség bizonyítása

1. Mohó választási tulajdonság - Strukturális csereérvelés

Állítás: Létezik optimális megoldás, amelyben a két legrövidebb darab **testvérek** a darabolási fában (azaz ugyanabból a vágásból származnak).

Indoklás: kivágás - beillesztéssel

- Vegyünk egy optimális fát.
- A legmélyebb szinten biztosan van két testvérlevél.
- Ha ezek nem a két legrövidebb darabhoz tartoznak, cseréljük ki őket a két legrövidebbre.
- A csere után a hosszabb darabok kevesebbszer szerepelnek költségként.

A csere nem növeli az összköltséget, így létezik optimális megoldás a mohó választással.

Stick Division — Redukált probléma

2. Optimális részprobléma tulajdonság

A mohó választás után a két legrövidebb darabot egyetlen

$$d = d_i + d_j$$

hosszúságú darabként kezelhetjük.

A fennmaradó feladat:

- ugyanilyen szerkezetű,
- eggyel kevesebb levéllel,
- a költségek összeadódnak.

Ha a redukált feladatra optimális fát építünk, és ehhez hozzávesszük a most létrehozott belső csúcsot, akkor az eredeti feladatra is optimális megoldást kapunk.

Ezért az algoritmus rekurzívan alkalmazható:

- mindig a két legrövidebb elemet vonjuk össze,
- az összegüket visszatesszük a halmazba.

Huffman-kódolás — Az alapfeladat

Adott n különböző karakter, mindegyikhez tartozik egy gyakoriság:

$$f_1, f_2, \ldots, f_n.$$

Feladat: Adjunk meg egy **prefixmentes bináris kódot** a karakterekhez úgy, hogy a

$$\sum f_i \cdot \ell_i$$

érték minimális legyen, ahol ℓ_i a karakter kódhosszát jelenti.

Prefixmentes kód:

- Egy kód *prefixmentes*, ha egyik kód sem a másik kezdete.
- Példa nem prefixmentes: 0, 01 (az 0 prefixe az 01-nek)
- Példa prefixmentes: 0, 10, 11
- Előny: így a dekódolás **egyértelmű** és mindig balról jobbra történhet.

Értelmezés:

- minden karakter egy levél,
- a kódfa bináris fa,
- a gyakoribb karaktereknek rövidebb kódot szeretnénk.

Stick Division és Huffman-kódolás

A Stick Division feladat matematikailag **ekvivalens a Huffman-kódolással**.

Megfeleltetés:

- Darab hossza $\leftrightarrow$ karakter gyakorisága
- Darabolási fa $\leftrightarrow$ kódfa
- Belső csúcs súlya $\leftrightarrow$ kódhossz-növekedés
- Összköltség $\leftrightarrow$ teljes kódolt hossz

A Huffman-algoritmus pontosan ugyanazt a mohó lépést használja:

- mindig a két legkisebb súlyú levelet vonja össze,
- így minimális súlyozott úthosszt kap.

Ez a mohó algoritmusok egyik legfontosabb klasszikus példája.

Stick Division — Kód

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int main() {
5      int x, n; cin >> x >> n;
6      priority_queue<int, vector<int>, greater<int>> pq;
7      for (int i = 0; i < n; ++i) {
8          int d; cin >> d;
9          pq.push(d);
10     }
11     long long cost = 0;
12     while (pq.size() > 1) {
13         int a = pq.top(); pq.pop();
14         int b = pq.top(); pq.pop();
15         cost += a + b;
16         pq.push(a + b);
17     }
18     cout << cost << '\n';
19 }
```

Stick Division — Csatlakozó kérdés

Ha a minimális összköltségű vágásokkal a lehető leghamarabb akarunk végezni, de párhuzamosan végezhetünk vágásokat több vágógéppel, akkor minimum hány vágógépre van szükségünk? (Minden vágás egy percig tart.)

Stick Division — Csatlakozó kérdés

Ha a minimális összköltségű vágásokkal a lehető leghamarabb akarunk végezni, de párhuzamosan végezhetünk vágásokat több vágógéppel, akkor minimum hány vágógépre van szükségünk? (Minden vágás egy percig tart.)

Megoldás:

- Építsük fel a vágások fáját, a minimális idő a legmélyebben lévő levelek mélysége.
- Minden vágásnak határozzuk meg a mélységét, illetve minden mélységhez számoljuk meg, hogy hány vágást kell végezni azon a mélységen.
- Kezdetben legyen egy vágógépünk.
- A mélységeken visszafelé haladva összegezzük mindig a vágások számát, és ha ez több lenne, mint a gépek száma + hátralévő percek száma, akkor növeljük a gépek számát, amíg elég nem lesz.

Stick Division — Csatlakozó kérdés: Kód

```
int x, n; cin >> x >> n;
using pii = pair<int, int>;
priority_queue<pii, vector<pii>, greater<pii>> pq;
for (int i = 0; i < n; ++i) {
    int d; cin >> d;
    pq.push({d, 0});
}
vector<vector<int>> g(n);
int node = n - 1;
while (pq.size() > 1) {
    auto [a, u] = pq.top(); pq.pop();
    auto [b, v] = pq.top(); pq.pop();
    if (u != 0) g[node].push_back(u);
    if (v != 0) g[node].push_back(v);
    pq.push({a + b, node--});
}
vector<int> cnt(n);
int maxdepth = 0;
std::function<void(int, int)> dfs = [&](int x, int d) {
    cnt[d]++;
    maxdepth = max(maxdepth, d);
    for (int i : g[x]) dfs(i, d + 1);
};
dfs(1, 1);
int sum = 0, machines = 1, time = 0;
for (int d = maxdepth; d > 0; d--) {
    time++;
    sum += cnt[d];
    while (sum > machines * time) machines++;
}
cout << machines << endl;
```

Potions (Codeforces)

Adott n $(1 \leq n \leq 200\,000)$ ital egymás után egy sorban:

$$a_1, a_2, \ldots, a_n \qquad (-10^9 \leq a_i \leq 10^9).$$

Kezdetben az életerőd 0. Balról jobbra haladsz, és minden italnál eldöntheted:

- megiszod, vagy
- kihagyod.

Megkötés: Az életerőd **soha nem lehet negatív**.

Feladat: Határozd meg, hogy **legfeljebb hány italt** tudsz meginni!

Példa:

4, −4, 1, −3, 1, −3

Egy optimális választás: 1., 3., 4., 5., 6. ital $\Rightarrow$ 5 darab

Potions — Mohó ötlet

Alapgondolat:

- haladjunk balról jobbra,
- próbáljunk meg *minél több italt* elfogadni.

Ideiglenes stratégia:

- minden italt megiszunk,
- nyilvántartjuk az eddig megivottakat.

Ha az életerő negatívvá válna:

- visszavonjuk **a legrosszabb eddigi döntést**,
- azaz eldobjuk a **legnegatívabb** megivott italt - ez lehet akár az éppen aktuális ital - így biztos nemnegatív lesz az életerőnk.

Egy új gondolat jelent meg, ami a korábbi mohó megoldásokban nem volt:

korábbi döntések visszavonása.

Potions — Helyesség bizonyítása

Előnyben maradás (Staying Ahead Argument)

Vizsgáljuk az italok első `k` elemét.

Állítás: Az algoritmus által kiválasztott italhalmaz

- maximális elemszámú az első `k` italból,
- és minden ilyen elemszámú megoldás közül **maximális életerőt** hagy maga után.

Indoklás:

- Ha az életerő nem negatív, minden eddigi döntés megengedett.
- Ha az életerő negatívvá válna, akkor *bármely érvényes megoldásnak* el kell hagynia legalább egy eddig megivott italt.
- A legnegatívabb ital elhagyása növeli legjobban az életerőt, így minden más választásnál „előnyösebb állapotot” hagy maga után.

Ezért minden lépés után az algoritmus legalább olyan jó állapotban van, mint bármely más részleges megoldás.

Potions — Algoritmus

Fenntartunk:

- egy változót az aktuális életerőre,
- egy halmazt a megivott italok értékeiből.

Balról jobbra:

1. hozzáadjuk a_i-t az életerőhöz,
2. betesszük a_i-t a halmazba,
3. ha az életerő negatív:
    - kivesszük a legkisebb elemet a halmazból,
    - visszavonjuk annak hatását.

A végén a halmaz mérete adja a választ.

Megvalósítás: **prioritási sor (min-heap)**.

Potions - Kód

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int main() {
5    priority_queue<int> q;
6    int n; cin >> n;
7    long long sum = 0;
8    for (int i = 0; i < n; i++) {
9      int x; cin >> x;
10     sum += x;
11     q.push(-x);
12     if (sum < 0) {
13       sum += q.top();
14       q.pop();
15     }
16   }
17   cout << q.size() << "\n";
18 }
```

Potions — Tanulság

Ez a feladat egy **új mohó mintát** mutat:

Elfogadás + visszavonás

Jellemzők:

- ideiglenesen mindent elfogadunk,
- megsértett feltételnél javítunk,
- mindig a „legrosszabb” döntést vonjuk vissza.

Ez a minta több feladatban megjelenik, ahol **prefixfeltételeket** kell betartani.

Fénykép (Mester)

Egy rendezvényre $\mathtt{N}$ ($1 \leq \mathtt{N} \leq 200\,000$) vendéget hívtak meg. Minden vendég egy időintervallummal adja meg, hogy mettől meddig lesz jelen:

$$(\mathtt{E_i}, \mathtt{T_i}) \qquad (1 \leq \mathtt{E_i} < \mathtt{T_i} \leq 10^9)\,.$$

Ha egy fényképet az $\mathtt{x}$ időpontban készítünk és

$$\mathtt{E_i} \leq \mathtt{x} < \mathtt{T_i},$$

akkor az adott vendég szerepel a képen.

Feladat:

- készítsünk **minél kevesebb** fényképet,
- úgy, hogy minden vendég szerepeljen legalább egy képen,
- és adjuk meg a fényképezés időpontjait is.

Példa:

Vendégek: (2, 4), (1, 4), (2, 7), (7, 13), (5, 10), (3, 9)

Megoldás: 2 fénykép, időpontok: 3, 9

Fénykép — Modellezés

Másképp megfogalmazva:

- adottak intervallumok,
- pontokat kell választani,
- minden intervallum tartalmazzon legalább egy kiválasztott pontot.

Ez a klasszikus **intervallum lefedése pontokkal** feladat.

Fontos:

- nem az intervallumokat választjuk ki,
- hanem a **fényképezés időpontjait**.

Ha mohó megoldással próbáljuk, hogy legyen a mohó választás?

Ötlet: balról jobbra nézzük. Mikor készüljön az első fénykép?

Fénykép — Mohó stratégia

Mohó választás:

- Rendezzük a vendégeket növekvő **távozási idő** szerint.
- Vegyük azt a vendéget, aki a **leghamarabb távozik**.
- Készítsünk fényképet pontosan az ő távozási idejénél *egy pillanattal korábban*.

Ezután az összes olyen vendéget kihagyjuk, aki ezen a fényképen szerepelt, és ismételjük a folyamatot.

Az így választott első fénykép:

- a lehető legkésőbbi időpontban készülő első fénykép,
- tehát maximalizálja az első fényképen lévő emberek számát.

Fénykép — Helyesség bizonyítása

1. Mohó választási tulajdonság

Indoklás: Kivágás–beillesztés (Exchange Argument)

Legyen G az a vendég, aki a leghamarabb távozik, és T_G az ő távozási ideje. **Állítás:** Létezik optimális megoldás, amely tartalmaz egy fényképet az

$$x = T_G - \varepsilon$$

időpontban.

Indoklás:

- Vegyünk egy optimális megoldást, és nézzük meg azt a fényképet, amelyen G szerepel.
- Ennek időpontja $x \leq T_G$.
- Ha ezt a képet x-ről $T_G - \varepsilon$-re „eltoljuk”:
  - G továbbra is rajta marad,
  - minden más vendég, aki rajta volt, szintén rajta marad.

A megoldás nem romlik, de illeszkedik a mohó választáshoz.

Fénykép — Helyesség bizonyítása

2. Optimális részprobléma tulajdonság

A mohó választás után:

- minden vendéget eltávolítunk, aki rajta van az elkészített képen,
- a fennmaradó vendégek időintervallumai változatlanok.

A fennmaradó feladat:

- ugyanolyan szerkezetű,
- csak kevesebb vendéggel.

Ezért:

- a mohó döntéssel kezdődő optimális megoldás kiegészíthető a redukált probléma optimális megoldásával,
- és így globálisan optimális megoldást kapunk.

Bónusz: Battle of Arrays

Alice és Bob felváltva játszanak (Alice kezd). Alice-nak egy $\mathtt{n}$ elemű, Bobnak egy $\mathtt{m}$ elemű pozitív egész tömbje van.

Egy lépésben a soron lévő játékos választ:

- egy $\mathtt{x}$ elemet a **saját** tömbjéből,
- egy $\mathtt{y}$ elemet az **ellenfél** tömbjéből.

Ezután:

- ha $\mathtt{y} \leq \mathtt{x}$, akkor $\mathtt{y}$ törlődik,
- ha $\mathtt{y} > \mathtt{x}$, akkor $\mathtt{y} \leftarrow \mathtt{y} - \mathtt{x}$.

Győzelem: aki a lépése után kiüríti az ellenfél tömbjét.

Feladat: optimális játék mellett döntsd el, ki nyer: **Alice vagy Bob**.

Korlátok:

$$1 \leq \mathtt{n}, \mathtt{m} \leq 10^5, \quad 1 \leq \mathtt{a_i}, \mathtt{b_i} \leq 10^9$$

Kimenet: tesztesetenként egy szó: `Alice` vagy Bob.

Battle of Arrays - Megoldás

Mohó stratégia:

- Az aktuális játékos mindig a saját legnagyobb elemével támadja az ellenfél legnagyobb elemét.

Megvalósítás:

- két prioritási sorral leszimuláljuk.

Helyesség bizonyítása?

Battle of Arrays - Megoldás

Mohó stratégia:

- Az aktuális játékos mindig a saját legnagyobb elemével támadja az ellenfél legnagyobb elemét.

Megvalósítás:

- két prioritási sorral leszimuláljuk.

Helyesség bizonyítása? Ellenpélda!

`a = (3, 3, 4)  b = (1, 5, 6)`

Ha Alice a $4 \to 6$ lépést teszi, akkor veszít, de a $4 \to 5$ lépéssel indulva nyerhet.

Történet:

- Az ICPC NERC 2025 versenyen így tűzték ki, és mindenki az elvi hibás megoldással oldotta meg.
- A verseny utáni napon találták az ellenpéldát.
- A bizottság utólag azt írta, hogy mivel mindenki úgy értette a feladatot, hogy a játékos csak a saját tömbjéből választhat és az ellenfél maximumát kell támadni, ezért úgy tekintenek rá, mint egy elírás a feladatszövegben.

Mohó algoritmusok — Minták és bizonyítások

Feladat	**Mohó ötlet**	**Bizonyítás típusa**
Movie Festival	Legkorábban befejeződő intervallum	Kivágás–beillesztés (Exchange Argument)
Towers	Legkisebb még nagyobb tetejű torony	Előnyben maradás (Staying Ahead)
Tasks and Deadlines	Rövidebb feladat előbb	Javító csere (Exchange Improvement)
Room Allocation	Ha lehet, használjunk meglévő szobát	Alsó korlát, előnyben maradás
Stick Division / Huffman-kód	Két legkisebb összekapcsolása	Strukturális csereérvelés
Potions	Elfogadás + visszavonás	Előnyben maradás
Fénykép	Legkorábban befejeződő intervallum	Kivágás–beillesztés

Tanulságok és tanácsok

1. A mohó algoritmus nem egy konkrét módszer
Hanem gondolkodási stílus:

- döntések sorozata,
- lokálisan optimális választások,
- globális helyesség bizonyítása.

2. A bizonyítás sokszor fontosabb, mint az algoritmus
Ha nem tudod megindokolni, miért jó:

- valószínűleg nem jó - keress ellenpéldát,
- vagy nem érted még eléggé.

3. A bizonyítás gyakori felépítése

- Mohó választási tulajdonság: feltehetjük, hogy az optimális megoldásnak része a választás
- Optimális részprobléma tulajdonság: a fennmaradó feladat ugyanolyan szerkezetű

Ezért érdemes struktúrában gondolkodni, például: balról jobbra felépítés, állapotok és lehetséges átmenetek, fa szerkezet, a folyamat megfordítása, stb.

Feladatok

Önálló munkára

Legnagyobb hasznú munkák - Feladat

`https://mester.inf.elte.hu/` → Mohó algoritmusok / 24.

Adott N munka $(1 \leq N \leq 10\,000)$. Minden munka 1 napot vesz igénybe, egy nap csak egy munkát lehet elvégezni. Minden munkának ismert a határideje H_i $(1 \leq H_i \leq 10\,000)$ és a haszna P_i $(1 \leq P_i \leq 1000)$.

Feladat: Válaszd ki a munkákat úgy, hogy a teljes összhaszon maximális legyen.

Bemenet:
Első sor: N. Következő N sor: minden sorban H_i P_i.

Kimenet:
Első sor: a maximális összhaszon.
Második sor: a kiválasztott munkák sorszámai az elvégzés sorrendjében.

Példa bemenet:

```
6
3 60
4 40
1 10
3 30
7 70
4 20
```

Kimenet:

```
220
6 4 1 2 5
```

Banquet Preparations 2 – Feladat

https://codeforces.com/contest/1607/problem/H

Adott n étel, a i-edik ételben a_i gramm hal és b_i gramm hús van. A kóstoló pontosan m_i grammot eszik az i-edik ételből ($0 \leq m_i \leq a_i + b_i$), szabadon eldöntve, mennyit eszik halból (x_i) és húsból (y_i).

Két étel azonosnak számít, ha az elfogyasztás után

$$(a_i - x_i,\ b_i - y_i)$$

párosuk megegyezik. A **változatosság** az eltérő ételek száma – ezt kell minimalizálni.

Cél: válaszd meg x_i, y_i értékeket ($x_i + y_i = m_i$), hogy a változatosság minimális legyen.

Korlátok:

$$1 \leq n \leq 2 \cdot 10^5, \quad 0 \leq a_i, b_i \leq 10^6$$

Kimenet:

- minimális változatosság,
- egy tetszőleges optimális megoldás.

Ütemezés - Feladat

`https://mester.inf.elte.hu/` → Mohó algoritmusok / 57.

Egy N napos $(2 \leq N \leq 10\,000)$ rendezvényre adott M potenciális esemény $(1 \leq M \leq 100\,000)$. Minden eseményre ismert: V_i időtartama és H_i határideje $(1 \leq V_i \leq H_i \leq N)$. Egyszerre csak egy esemény tartható.

Feladat: Válaszd ki a lehető legtöbb eseményt és rendelj hozzá kezdőnapot úgy, hogy minden esemény a határidőn belül befejeződjön.

Bemenet:
Első sor: N M.
Következő M sor: minden sorban V_i H_i, rendezve H_i, majd V_i szerint.

Kimenet:
Első sor: a legtöbb megtartható esemény K száma
Következő K sor: minden sor az esemény sorszámát és kezdőnapját tartalmazza

Példa bemenet:

```
10 7
2 3
4 5
2 7
2 8
3 8
1 10
4 10
```

Kimenet:

```
4
1 1
3 3
4 5
6 7
```

Halve or Subtract – Feladat

https://codeforces.com/contest/1799/problem/F

Adott egy n hosszú pozitív egész tömb $a_1, \ldots, a_n$ és egy b konstans. Műveletek tetszőleges sorrendben végezhetők:

- **Felezés:** $a_i \leftarrow \lceil a_i/2 \rceil$ (legfeljebb k_1 alkalom összesen),
- **Csökkentés:** $a_i \leftarrow \max(a_i - b, 0)$ (legfeljebb k_2 alkalom összesen).

Minden indexre mindkét művelet **legfeljebb egyszer** alkalmazható.

Cél: a műveletek optimális kiválasztásával minimalizálni $\sum_{i=1}^{n} a_i$ értékét.

Korlátok:

$$1 \leq n \leq 5000, \quad 1 \leq a_i, b \leq 10^9, \quad 0 \leq k_1, k_2 \leq n$$

Kimenet: tesztesetenként egy szám – a minimálisan elérhető összeg.

Múzeum - Feladat

`https://mester.inf.elte.hu/` → Mohó algoritmusok / 58.

Adott egy $\mathtt{N} \times \mathtt{N}$ méretű múzeum $(1 \leq \mathtt{N} < 10\,000)$. Minden teremből nyílik ajtó a négy szomszédos terembe. Ha egy őr egy teremben áll, akkor a négy irányban levő összes termet belátja. Minden őr megad egy **téglalap alakú területet**, amelyen belül szeretne elhelyezkedni.

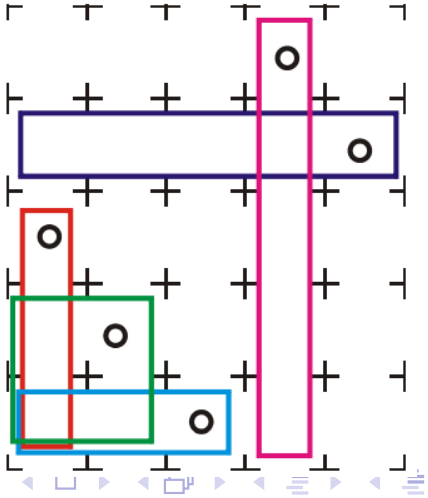
Feladat: Helyezzük el az őröket úgy, hogy az összes termet belássák, az őrök **ne lássák egymást**, és minden őr a megadott téglalapján belül legyen.

Bemenet:
Első sor: $\mathtt{N}$. Következő $\mathtt{N}$ sor: egy őr téglalapja (bal alsó és jobb felső sarok).

Kimenet:
$\mathtt{N}$ sor: az őrök elhelyezése (oszlop, sor). Ha nincs megoldás: -1.

Példa bemenet:

```
5
1 1 3 1
1 1 1 3
1 1 2 2
4 1 4 5
1 4 5 4
```

Kimenet:

```
3 1
1 3
2 2
5 4
4 5
```